CSA17 - Constraint-Based Problem-Solving Report

Artificial Intelligence - Assessment Tool 2

Q1. Hospital Doctor Shift Scheduling Problem Explanation

Three doctors (D1, D2, D3) must each get exactly one of three shifts (Morning, Afternoon, Night) subject to five constraints, including D1 never working Night, D3 never working Morning, and D2's shift preceding D3's shift in the day.

Approach

Backtracking Search with Forward Checking and Minimum-Remaining-Values (MRV) ordering. MRV always assigns the most constrained doctor next; forward checking prunes every other doctor's domain immediately after each assignment, so illegal branches are eliminated before they are ever attempted.

Python Code

```
def build_initial_domains():
    domains = {d: {"Morning", "Afternoon", "Night"} for d in DOCTORS}
    domains["D1"].discard("Night")    # D1 cannot work Night
    domains["D3"].discard("Morning")          # D3 cannot work Morning
    return domains

def forward_check(domains, assigned_doctor, assigned_shift):
    new_domains = {d: set(v) for d, v in domains.items()}
    for d in DOCTORS: if d != assigned_doctor:
    new_domains[d].discard(assigned_shift)
    if assigned_doctor == "D2":
        new_domains["D3"] = {s for s in new_domains["D3"] if
                        SHIFT_ORDER[s] > SHIFT_ORDER[assigned_shift]}
    if assigned_doctor == "D3":
        new_domains["D2"] = {s for s in new_domains["D2"] if
                        SHIFT_ORDER[s] < SHIFT_ORDER[assigned_shift]}
    return new_domains

def select_unassigned_variable(domains, assignment):
    unassigned = [d for d in DOCTORS if d not in assignment]
    return min(unassigned, key=lambda d: len(domains[d]))  # MRV

def backtracking_csp(assignment, domains, trace):
    if len(assignment) == len(DOCTORS):
        return dict(assignment)
    var = select_unassigned_variable(domains, assignment) for shift in
    sorted(domains[var], key=lambda s: SHIFT_ORDER[s]):
    assignment[var] = shift
```

```
    pruned = forward_check(domains, var, shift) if all(len(pruned[d]) > 0
    for d in DOCTORS if d not in assignment):
        result = backtracking_csp(assignment, pruned, trace) if
        result is not None:
            return result
    del assignment[var] return
None
```

Output

Trying D1 = Morning → Trying D2 = Afternoon → Trying D3 = Night. Final valid schedule: D1 = Morning, D2 = Afternoon, D3 = Night, reached with zero rejections/backtracks thanks to MRV + forward checking (see A2_OUTPUT_AI.py for the full trace).

Q2. Robot Grid Path Planning (5×5 Grid) Problem Explanation

A robot must travel from S = (0,0) to G = (4,4) on a 5×5 grid using Up/Down/Left/Right moves of cost 1, avoiding a set of obstacle cells arranged as an irregular wall through the middle of the grid.

Approach

Breadth-First Search using a FIFO deque as the frontier (popleft to expand, append to add). The Manhattan distance h(n) is printed at every expansion purely for analysis and does not affect expansion order; since all move costs are equal, plain BFS already guarantees the optimal path.

Python Code

```
def bfs_with_frontier_log(start, goal,
    grid): frontier = deque([start])
    came_from = {start: None}
    visited =
    {start} step = 0
    while frontier:
    step += 1
        current = frontier.popleft()
        print(f"Expand step {step}: node={current}, "
              f"h(n)={manhattan(current, goal)}, frontier_before={list(frontier)}")
        if current == goal:
            break
        for dx, dy in [(0, 1), (1, 0), (0, -1), (-1, 0)]:
            nxt = (current[0] + dx, current[1] + dy)
            r, c = nxt if 0 <= r < 5 and 0 <= c < 5 and grid[r][c] == 0 and nxt not
            in visited: visited.add(nxt) came_from[nxt] = current
            frontier.append(nxt)
    # reconstruct path via came_from pointers
    ...
```

Output

Optimal path: (0,0) → (0,1) → (0,2) → (1,2) → (2,2) → (2,3) → (2,4) → (3,4) → (4,4). Cost: 8.

Q3. Autonomous Rescue Robot (Online Search Agent) Problem Explanation

A rescue robot must reach a survivor at G from S in a grid where some cells are blocked and some are risky (extra cost 2), sensing only adjacent cells and updating its plan as it moves (partial observability, online search).

Approach

An Online Search Agent repeatedly senses its neighbours, updates its known map, and replans the remaining route with Uniform Cost Search restricted to known cells, before committing to just one step — the sense-plan-act cycle that lets it react to obstacles and risk as it discovers them. See Q3(a)-(d) in the Solution document for the full constraint analysis, AI-concept mapping and justification.

Python Code

```
def online_ucs_rescue(start, goal, environment):
   known_map = {start: environment[start[0]][start[1]]}
   current, total_cost, travelled = start, 0, [start]
   while current != goal: sensed =
   sense_adjacent(current, environment)
   known_map.update(sensed)
      # UCS re-plan restricted to known_map only
      frontier = [(0, current)] dist, parent, visited = {current:
      0}, {current: None}, set() while frontier:
         cost, node =
         heapq.heappop(frontier) if node in
         visited: continue
         visited.add(node)
         if node == goal:
            break
         for dx, dy in [(0, 1), (1, 0), (0, -1), (-1, 0)]: neighbour =
            (node[0] + dx, node[1] + dy) if
            known_map.get(neighbour, 0) != 1: step_cost =
            cell_cost(known_map.get(neighbour, 0)) new_cost =
            cost + step_cost if neighbour not in dist or new_cost <
            dist[neighbour]: dist[neighbour] = new_cost
            parent[neighbour] = node
                  heapq.heappush(frontier, (new_cost, neighbour))
      # take only the first step of the freshly planned route
      next_step = reconstruct(parent, goal)[1]
      total_cost += cell_cost(known_map.get(next_step,
   0)) current = next_step travelled.append(current) return
   travelled, total_cost
```

Output

Least-cost path: (0,0) → (1,0) → (2,0) → (2,1) → (2,2) → (3,2) → (3,3) → (3,4) → (4,4). Total cost: 8 — the online re-planning steered the robot around every risky cell it sensed, so no risk penalty was incurred.